

Netty学以致用 之



开发“一问一答”复读机程序

北京理工大学计算机学院
金旭亮

概述



前面的课程中，我们已经介绍如何在客户端与服务端之间相互发送信息，但给出的示例过于简单，并没有足够的灵活性。



本讲将在前面课程所介绍的知识与技术基础之上，构建一个典型的在客户端与服务器应用之间相互交换信息的示例——Echo应用程序。



这个程序的工作过程是这样的：

- 用户从键盘不断地输入信息，发给服务端。
- 服务端原封不动地返回。
- 当需要退出时，客户端输入`exit`命令即可。

服务端Handler

```
@ChannelHandler.Sharable
public class EchoServerHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        ByteBuf in = (ByteBuf) msg;
        String clientMessage = in.toString(CharsetUtil.UTF_8);
        System.out.println("服务器收到: " + clientMessage);
        if (clientMessage.equals("exit")) {
            System.out.println("断开连接: " + ctx.channel().remoteAddress());
            ctx.writeAndFlush(Unpooled.copiedBuffer(clientMessage, CharsetUtil.UTF_8));
            ctx.close();
        } else {
            String returnMessage = "[鹦鹉学舌服务器]" + clientMessage;
            ctx.writeAndFlush(Unpooled.wrappedBuffer(
                returnMessage.getBytes(StandardCharsets.UTF_8)));
        }
        //释放掉ByteRef
        ReferenceCountUtil.release(msg);
    }
}
```

服务端主程序-1

```
public class EchoServer {  
    private final int port; 2 usages  
  
    public EchoServer(int port) { this.port = port; }  
  
    public static void main(String[] args) throws Exception {  
        new EchoServer(port: 9000).start();  
    }  
  
    public void start() throws Exception {...}  
}
```

代码详见下页PPT

服务端主程序-2

```
public void start() throws Exception {
    final EchoServerHandler serverHandler = new EchoServerHandler();
    EventLoopGroup boss = new NioEventLoopGroup();
    EventLoopGroup worker = new NioEventLoopGroup();
    try {
        ServerBootstrap boot = new ServerBootstrap();
        boot.group(boss, worker)
            .channel(NioServerSocketChannel.class)
            .localAddress(new InetSocketAddress(port))
            .childHandler(new ChannelInitializer<SocketChannel>() {
                @Override
                protected void initChannel(SocketChannel ch) throws Exception {
                    ch.pipeline().addLast(serverHandler);
                }
            });
        ChannelFuture f = boot.bind().sync();
        System.out.println("鸚鵡學舌服務器已經就緒。");
        f.channel().closeFuture().sync();
    } finally {
        boss.shutdownGracefully().sync();
        worker.shutdownGracefully().sync();
    }
}
```

处理键盘输入

```
// 此类将等待用户在键盘输入一个字符串，然后敲回车
public class InputHelper {
    // 接收用户输入
    public static String getUserInput(String prompt) throws IOException {
        var bufferReader = new BufferedReader(new InputStreamReader(System.in));
        String userInput = "";
        while (true) {
            System.out.println(prompt);
            userInput = bufferReader.readLine();
            if (userInput.isBlank() || userInput.isEmpty()) {
                System.out.println("不能发空消息!");
                continue;
            }
            return userInput;
        }
    }
}
```

客户端Handler-1

```
//注意SimpleChannelInboundHandler会自动释放ByteBuf
public class EchoClientHandler extends SimpleChannelInboundHandler<ByteBuf> { 1 usage
    //接收服务端传回的消息
    @Override 1 usage
    protected void channelRead0(ChannelHandlerContext ctx, ByteBuf msg) throws Exception {
        String serverMessage = msg.toString(CharsetUtil.UTF_8);
        if (serverMessage.equals("exit")) {
            ctx.close();
            System.out.println("程序退出");
        } else {
            System.out.println(serverMessage);
        }
        //收到一条消息之后，再次等待用户输入
        handleUserInput(ctx);
    }
    //当成功连接上服务器时，让用户输入消息
    @Override
    public void channelActive(ChannelHandlerContext ctx) { handleUserInput(ctx); }
    //接收用户输入
    private static void handleUserInput(ChannelHandlerContext ctx) {...}
}
```

见下页PPT

客户端Handler-2

```
//当成功连接上服务器时, 让用户输入消息
@Override
public void channelActive(ChannelHandlerContext ctx) {
    handleUserInput(ctx);
}
//接收用户输入
private static void handleUserInput(ChannelHandlerContext ctx) {
    try {
        String userInput = InputHelper.getUserInput("请输入要发送给服务器的消息:");
        //将消息发给服务器
        ctx.writeAndFlush(Unpooled.copiedBuffer(userInput, CharsetUtil.UTF_8));
        if (userInput.equals("exit")) {
            System.out.println("关闭与服务器的连接, 退出客户端.....");
            ctx.close();
        }
    } catch (IOException e) {
        System.out.println(e.getMessage());
        ctx.close();
    }
}
```


客户端主程序

```
//用户输入字符串，服务端原样返回
public class EchoClient {
    private final String host; 2 usages
    private final int port; 2 usages

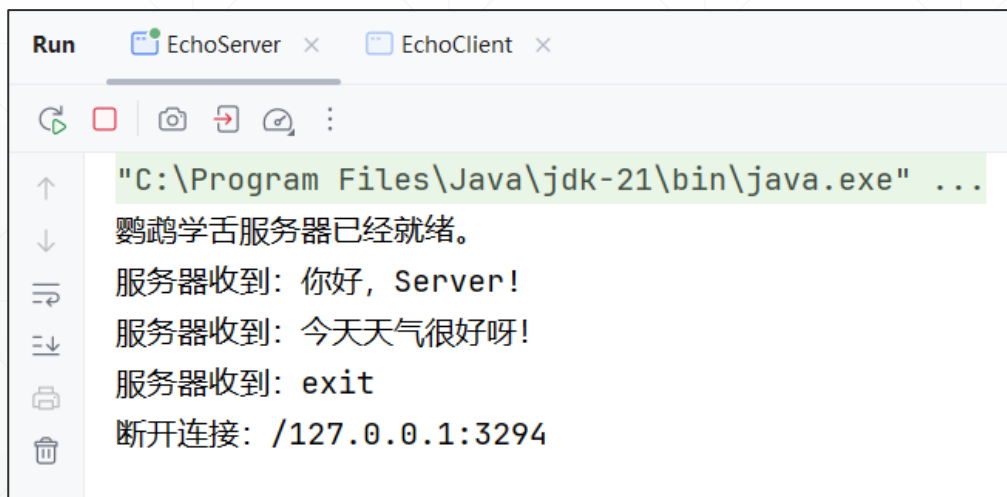
    public EchoClient(String host, int port) { 1 usage
        this.host = host;
        this.port = port;
    }

    public void start() throws Exception {...}

    public static void main(String[] args) throws Exception {
        new EchoClient(host: "127.0.0.1", port: 9000).start();
    }
}
```

运行截图

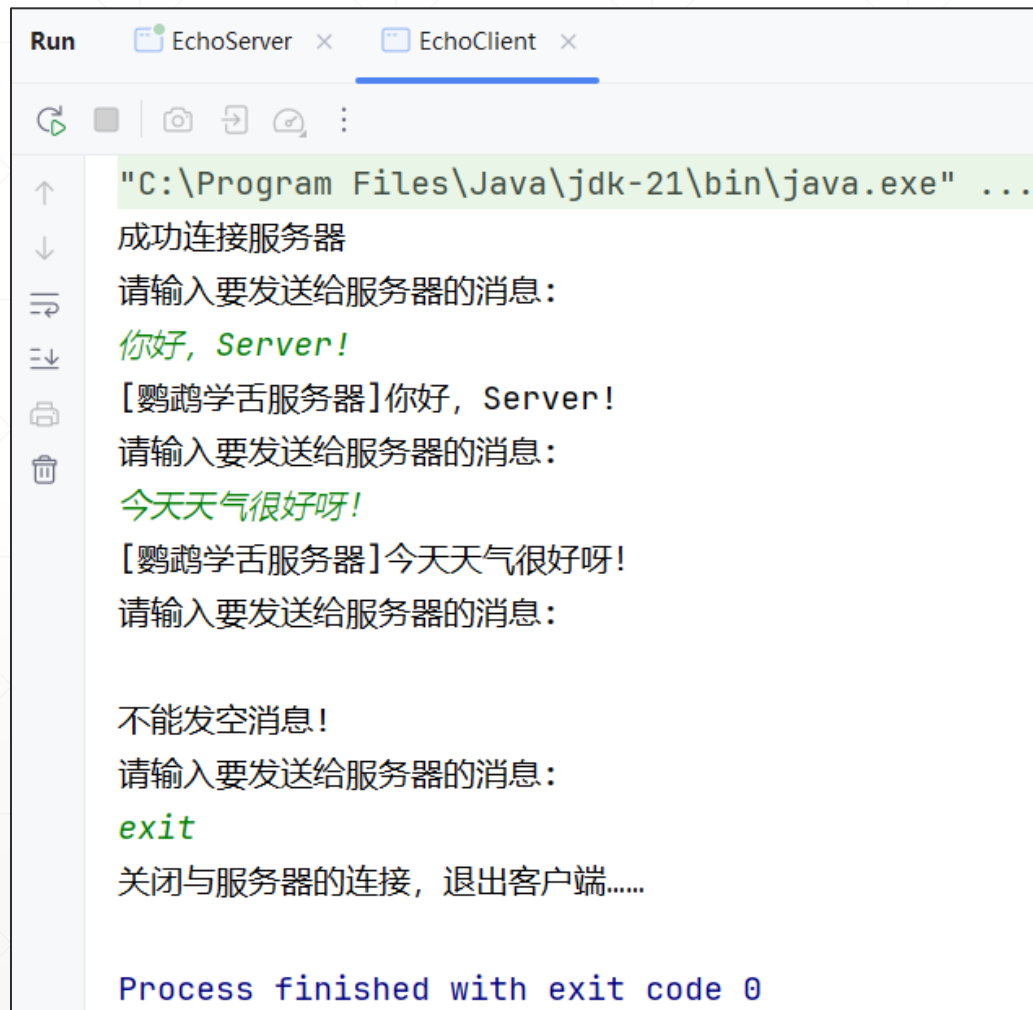
服务器



The screenshot shows the server console output. The title bar indicates two tabs: 'EchoServer' and 'EchoClient'. The console output is as follows:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
鸚鵡學舌服務器已經就緒。  
服務器收到: 你好, Server!  
服務器收到: 今天天氣很好呀!  
服務器收到: exit  
斷開連接: /127.0.0.1:3294
```

客戶端



The screenshot shows the client console output. The title bar indicates two tabs: 'EchoServer' and 'EchoClient'. The console output is as follows:

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
成功連接服務器  
請輸入要發送給服務器的消息:  
你好, Server!  
[鸚鵡學舌服務器]你好, Server!  
請輸入要發送給服務器的消息:  
今天天氣很好呀!  
[鸚鵡學舌服務器]今天天氣很好呀!  
請輸入要發送給服務器的消息:  
  
不能發空消息!  
請輸入要發送給服務器的消息:  
exit  
關閉與服務器的連接, 退出客戶端.....  
  
Process finished with exit code 0
```

小结



本讲通过实例，介绍了一个非常常见的“一问一答”网络应用程序开发的技术要点。



服务端没啥特殊的，客户端由于要完成三件事情：处理键盘输入、将用户输入的消息发送出去，接收服务端传回的信息，就比较复杂一点。请同学们仔细阅读示例源码，体会其执行流程。



如果在服务端部署一个AI模型，或者访问一个互联网上的公用AI服务，就可以轻松地开发出一个“聊天机器人”，这是一个很有意思的课题，感兴趣的同学可以自行探索和实践。